**Sistemas Multinúcleo e Distribuídos**
Mestrado em Engenharia Informática
Instituto Superior de Engenharia do Porto

# Histogram Equalization and Parallel Processing in Java

## Introduction

Digital image processing can significantly enhance the visual detail and clarity of a photograph. In many cases, a low-contrast image can be transformed into a much sharper one by redistributing the intensities of its pixels. This technique is known as **histogram equalization**. The algorithm converts the image to grayscale and spreads pixel intensities across as much of the available range as possible, resulting in increased contrast and improved visibility of features. For example, image 1 after being processed with histogram equalization results in image 2.

Figure 1: Original Image

The histogram of image 1 is presented in figure 3 and the histogram of image 2 is presented in figure 4. This transformation allows to enhance image sharpness considerably.

In this project, you will implement histogram equalization in Java, experimenting with sequential, multithreaded, and parallel implementations. You are encouraged to explore Java concurrency mechanisms such as threads, thread pools, atomic operations, parallel streams, and the Fork/Join framework. The goal is not only correctness but also an exploration of performance, speedup, and hardware behavior under parallel workloads.

Figure 2: Processed Image

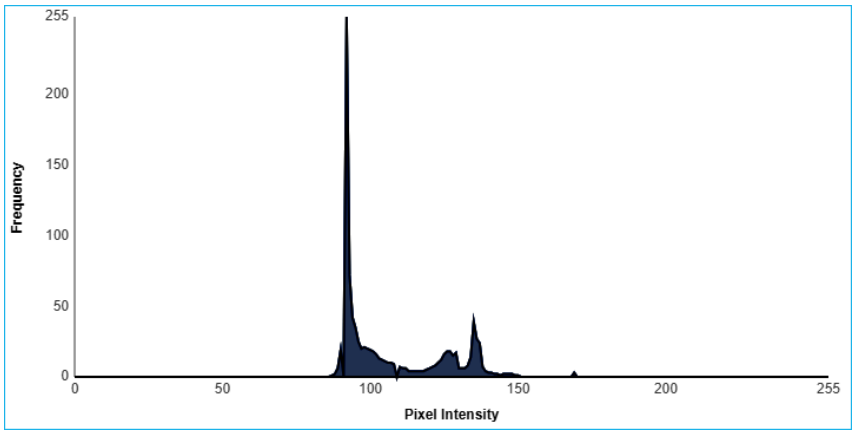


Figure 3: Histogram of original Image

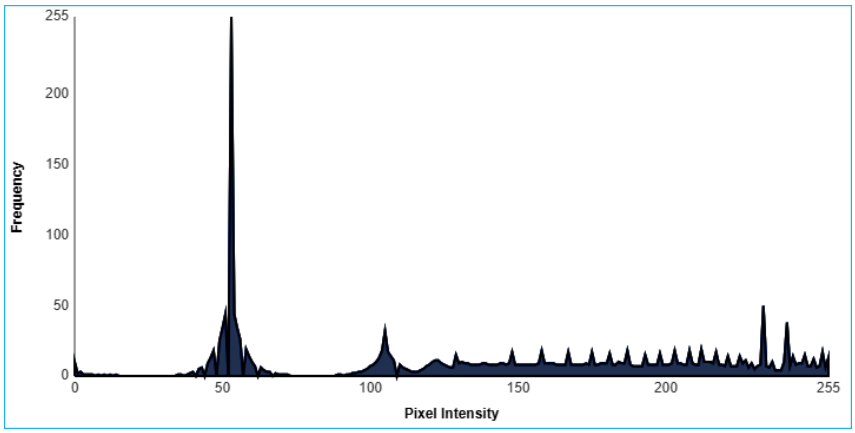


Figure 4: Histogram of processed Image

The algorithm for histogram equalization is divided into three main stages:

1. Compute the luminosity histogram of the source image
2. Compute the cumulative luminosity histogram
3. Rewrite each pixel using the cumulative histogram to increase contrast and generate the processed image.

Each stage is well-suited for parallelization and provides opportunities to experiment with different concurrency strategies.

# Computing the Luminosity Histogram

The first step is to compute the **luminosity** of each pixel. Luminosity is a standardized measure of perceived brightness based on a pixel's RGB components and ranges from 0 (black) to 255 (white). You may use the provided method:

```
int luminosity = computeLuminosity(red, green, blue);
```

The **luminosity histogram** is an array of 256 integers where index $i$ represents the number of pixels with luminosity $i$. For example:

- hist[0] = number of pixels with luminosity 0
- hist[1] = number of pixels with luminosity 1
- ...
- hist[255] = number of pixels with luminosity 255

Low-contrast images have histograms concentrated in a narrow range of values, while high-contrast images have histograms spread across the full intensity spectrum.

# Computing the Cumulative Luminosity Histogram

Next, compute the **cumulative histogram**, also an array of 256 integers, where entry $i$ represents the number of pixels with luminosity $\leq i$.

Example:

| Index | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Histogram | 1 | 3 | 5 | 7 | 9 | 11 |
| Cumulative | 1 | 4 | 9 | 16 | 25 | 36 |

This cumulative distribution will be used to map original luminosities to new values that cover a wider range, thus increasing contrast.

# Modifying Each Pixel to Increase Contrast

Once the cumulative histogram is available, each pixel's luminosity is transformed according to:

$$\text{newLuminosity} = \left\lfloor 255 \times \frac{\text{cumulativeHist}[L]}{\text{totalPixels}} \right\rfloor$$

where $L$ is the pixel's original luminosity.

Since grayscale pixels have equal R, G, and B values, each modified pixel becomes:

$$(R, G, B) = (\text{newLuminosity}, \text{newLuminosity}, \text{newLuminosity})$$

### Example

Suppose the image contains 10 pixels with luminosities between 125 and 130. The cumulative histogram might be:

| Luminosity | 125 | 126 | 127 | 128 | 129 | 130 |
|---|---|---|---|---|---|---|
| Cumulative | 1 | 3 | 5 | 7 | 9 | 10 |

- A pixel with luminosity 125 has $1/10 = 10\%$ of pixels $\leq 125 \rightarrow$ new luminosity $\approx 25$
- A pixel with luminosity 129 has $9/10 = 90\%$ of pixels $\leq 129 \rightarrow$ new luminosity $\approx 229$

This stretches the narrow range 125 to 130 into a range like 25 to 229, producing much higher contrast. For further information and enhancements take a look to the book Digital Image Processing 4th edition [1] and the book Computer Science: An Interdisciplinary Approach [2] and the web site of the former book [1].

## Implementation Approaches

The project requires the development and analysis of the following implementations:

1. **Sequential Solution**: A baseline implementation that processes the data sequentially without any concurrency.
2. **Multithreaded Solution (Without Thread Pools)**: An explicit multithreaded implementation where threads are manually managed, focusing on work distribution and synchronization.
3. **Multithreaded Solution (With Thread Pools)**: An implementation utilizing thread pools to manage threads efficiently, reducing the overhead of thread creation and destruction.
4. **Fork/Join Framework Solution**: A solution leveraging the Fork/Join framework to recursively split tasks and join results, suitable for divide-and-conquer algorithms.
5. **CompletableFutures-Based Solution**: An asynchronous implementation using `CompletableFuture` to manage tasks and their dependencies without explicit thread management.

[1] `https://introcs.cs.princeton.edu/java/31datatype/`

6. **Garbage Collector Tuning**: Configure and tune Java garbage collectors to optimize memory management and improve application performance.

## 1. Sequential Solution

The sequential solution serves as a baseline for performance comparison. It processes the input file sequentially. This implementation helps in understanding the limitations of sequential processing in handling large data volumes on multicore systems.

Sequential processing is straightforward but doesn't utilize the available cores in a multicore system, leading to suboptimal performance. Establishing a baseline allows us to quantify the benefits of concurrent approaches.

## 2. Multithreaded Solution (Without Thread Pools)

In this approach, multiple threads are explicitly created and managed to process different parts of the data concurrently. A common strategy like the producer-consumer pattern can be employed for work distribution.

Manually managing threads provides fine-grained control over concurrency but introduces complexity in thread coordination and resource management. This approach helps in understanding the challenges of thread synchronization and data sharing.

## 3. Multithreaded Solution (With Thread Pools)

This implementation utilizes thread pools to manage threads more efficiently. The thread pool handles thread creation and reuse, reducing the overhead associated with thread lifecycle management.

Using thread pools simplifies thread management and can lead to better resource utilization. It allows for scalable solutions that can adjust the number of active threads based on the workload and system capabilities.

## 4. Fork/Join Framework Solution

The Fork/Join framework is used to implement a solution that recursively splits the task into subtasks, processes them in parallel, and combines the results.

The Fork/Join framework is designed for tasks that can be broken down into smaller, independent subtasks. It efficiently utilizes all available processors and is suitable for divide-and-conquer algorithms, potentially leading to significant performance improvements.

## 5. CompletableFutures-Based Solution

This approach uses `CompletableFuture` for asynchronous programming. It allows tasks to be executed in non-blocking ways and handles the composition of asynchronous operations.

`CompletableFuture` provides a high-level API for writing asynchronous code without manually managing threads. It simplifies handling of dependent tasks and can lead to more readable and maintainable code.

### 6. Garbage Collector Tuning

Java provides several garbage collectors that operate concurrently with applications to manage memory by reclaiming unused objects, employing different strategies to optimize performance. You should provide evidence of such configuration work, including documentation or logs demonstrating the configuration and tuning of the garbage collector used in your application, and clearly explain why you selected a particular garbage collector, referencing how it aligns with your application's performance characteristics and resource usage patterns. Additionally, analyze how the chosen garbage collector affects your application's performance and resource utilization, including any improvements observed.

## Concurrency and Synchronization

Appropriate synchronization mechanisms must be used to ensure thread safety and data consistency across all concurrent implementations. Choices of data structures (e.g., thread-safe collections) and synchronization techniques (e.g., locks and atomic variables) should be justified and their impact on performance analyzed.

## Generation of Results

Each implementation should measure and record:

- **Execution Time:** Total time taken to process the dataset.
- **Resource Utilization:** CPU and memory usage during execution.
- **Scalability Analysis:** Performance metrics as the size of the dataset or the number of threads changes.

Results should be presented using automatically generated tables and charts. The analysis should compare the performance of each implementation, discussing:

- **Efficiency Gains:** Improvements over the sequential baseline.
- **Scalability:** How performance scales with increased data size or additional cores.
- **Overhead Analysis:** Impact of thread management and synchronization on performance.
- **Bottlenecks:** Identification of any limitations or areas where performance does not improve as expected.

## Starter Code

Starter code is provided, including a simple image-handling API and an example showing how to apply histogram equalization sequentially. The API is summarized below:

`Color[][] loadImage(String filename)`: Given the path to an image file, this method returns a `Color[][]` array containing the pixels of that image.

`void writeImage(Color[][] image, String filename)`: Given a `Color[][]` array and an output file path, this method writes the pixel array to the specified file, producing an image on the filesystem.

`Color[][] copyImage(Color[][] image)`: Creates and returns a copy of the provided `Color[][]` image. This is useful when modifications should be made on a duplicate rather than the original data.

The file **ApplyFilter.java** demonstrates the use of this API. It creates a `HistogramFilter` object, loads an image from the filesystem, applies a basic histogram equalization to it, and then writes the processed image back to the filesystem.

# Report

The report should include the following content:

- **Cover**: Title, class identification, and authors (student numbers and full names)
- **Introduction**
- **Objectives**
- **Implementation Approaches**
  - Sequential Solution
  - Multithreaded Solution (Without Thread Pools)
  - Multithreaded Solution (With Thread Pools)
  - Fork/Join Framework Solution
  - CompletableFuture-Based Solution
  - Garbage Collector Tuning
- **Concurrency and Synchronization**
- **Performance Analysis**
- **Conclusions**

Each section should include a description of the implementation, justification of the chosen approach, and essential code snippets (avoid including all code).

# Grading

Grading is awarded following the criteria described next:

1. **Correctness and Functionality (60 points)**
   - Multithreaded Implementation without Thread Pools (15 points): Correctly implements multithreading. Manages threads and synchronization properly.

- Thread Pool Implementation (15 points): Effectively uses `ExecutorService` and thread pools. Optimizes thread pool size for performance.
- Fork/Join Framework Implementation (15 points): Successfully implements the Fork/Join approach with proper task splitting and result aggregation.
- CompletableFuture Implementation (10 points): Correctly uses `CompletableFuture` for asynchronous tasks. Handles task completion and combines results appropriately.
- Garbage Collector Tuning (5 points): Demonstrates effective configuration and tuning of garbage collectors. Provides evidence and justification for choices.

2. **Performance Analysis (30 points)**
   - Benchmarking (15 points): Measures execution time, CPU, memory usage, and garbage collection metrics. Tests with various data sizes and thread counts.
   - Analysis Report (15 points): Presents data using graphs or tables. Provides insights into why certain implementations perform better. Discusses challenges, including garbage collection effects, and solutions.
3. **Code Quality and Documentation (10 points)**
   - Code Organization (5 points): Follows best practices for code structure and naming conventions. Uses appropriate data structures and algorithms.
   - Documentation and Comments (5 points): Includes comments explaining complex sections. Provides a README file with setup and execution instructions.
4. **Understanding of Concepts during Presentation (Student Weighting)**
   - Has a very good understanding of the topic and can explain the developed code well: 100%
   - Has some shortcomings in understanding the topic or (only) in the developed code: 80%
   - Has some shortcomings in understanding both the topic and the developed code: 70%
   - Encounters significant difficulties in mastering the topic or (only) in the developed code: 60%
   - Encounters significant difficulties in mastering both the topic and the developed code: 50%
   - Demonstrates major difficulties in mastering the topic or (only) in the developed code: 40%
   - Faces major difficulties in fully understanding both the topic and the developed code: 25%
   - No-show or demonstrates a lack of knowledge in both the topic and the developed code: 0%

Student's grade is = Work grading * Student Weighting

# Scheduling

The final deadline for the submission of this project is the $12^{th}$ of May of 2026 and the presentations will take place on the following two weeks during TP and PL classes.

# Code of Honor

As per the "Código de boas Práticas de Conduta" dated October 27, 2020, students must submit a declaration as described in Article 8 for each project submission. This declaration is a signed commitment to the originality of the submitted work. Failure to submit this declaration will result in the work not being evaluated. Submitting work that violates the declaration will have legal consequences.

# References

[1] R.C. Gonzalez and R.E. Woods. *Digital Image Processing.* Pearson, 2018.

[2] Robert Sedgewick and Kevin Wayne. *Computer Science: An Interdisciplinary Approach.* Addison-Wesley Professional, 1st edition, 2016.